

SKEL 4223: Digital Signal Processing *with* MATLAB

CHAPTER 1

DISCRETE-TIME SIGNAL

Dr. Musa Mohd Mokji

1.0 Construct Sinusoidal Signal.

```

A = 1;           % Amplitude.
F = 100;         % Frequency in Hz.
D = 0.02;        % Signal duration in second.
Fs = 5000;       % Frequency Sampling (no. of samples in 1 second).
N = D*Fs;        % Number of samples for duration D.
n = 0:N;         % Discrete-time (sample).

x = A*sin(2*pi*(F/Fs)*n);

plot(n/Fs*1000,x,n/Fs*1000,x,'o');
legend('Continuous signal','Discrete Sample')
xlabel('time, ms')
ylabel('Amplitude')
titleString = sprintf('%.1f kHz Sine Signal',F/1000);
title(titleString)
grid on
    
```

1.1 Introduction

Continuous-time sinusoidal signals can be varied based on 3 parameters, which are amplitude A , frequency F and phase ϕ according to below equation:

$$x(t) = A \sin(2\pi F t + \phi)$$

For discrete-time signal, time t is sampled into n where $t = \frac{n}{F_s}$. Here F_s is the sampling frequency. Thus, the discrete-time signal can be written as the following:

$$x[n] = A \sin\left(2\pi \frac{F}{F_s} n + \phi\right)$$

1.2 Setting the Variables

First let set the continuous parameters. Here we set amplitude as 1, frequency 100Hz and the duration of the signal is $0 \leq t \leq 0.02\text{s}$.

```
A = 1;           % Amplitude.
F = 100;         % Frequency in Hz.
D = 0.02;        % Signal duration in second.
```

Then, perform the sampling process to the continuous-time t to obtain the discrete-time n . Let's set the frequency sampling as 5kHz . Then, the n can be computed based on $n = tF_s$. In this example we will convert a duration of $0 \leq t \leq 0.02\text{s}$ to discrete-time duration. Thus, $0F_s \leq n \leq PF_s$.

```
Fs = 5000;       % Frequency Sampling (no. of samples in 1 second).
N  = D*Fs;       % Number of samples for duration D.
n  = 0:N;        % Discrete-time (sample).
```

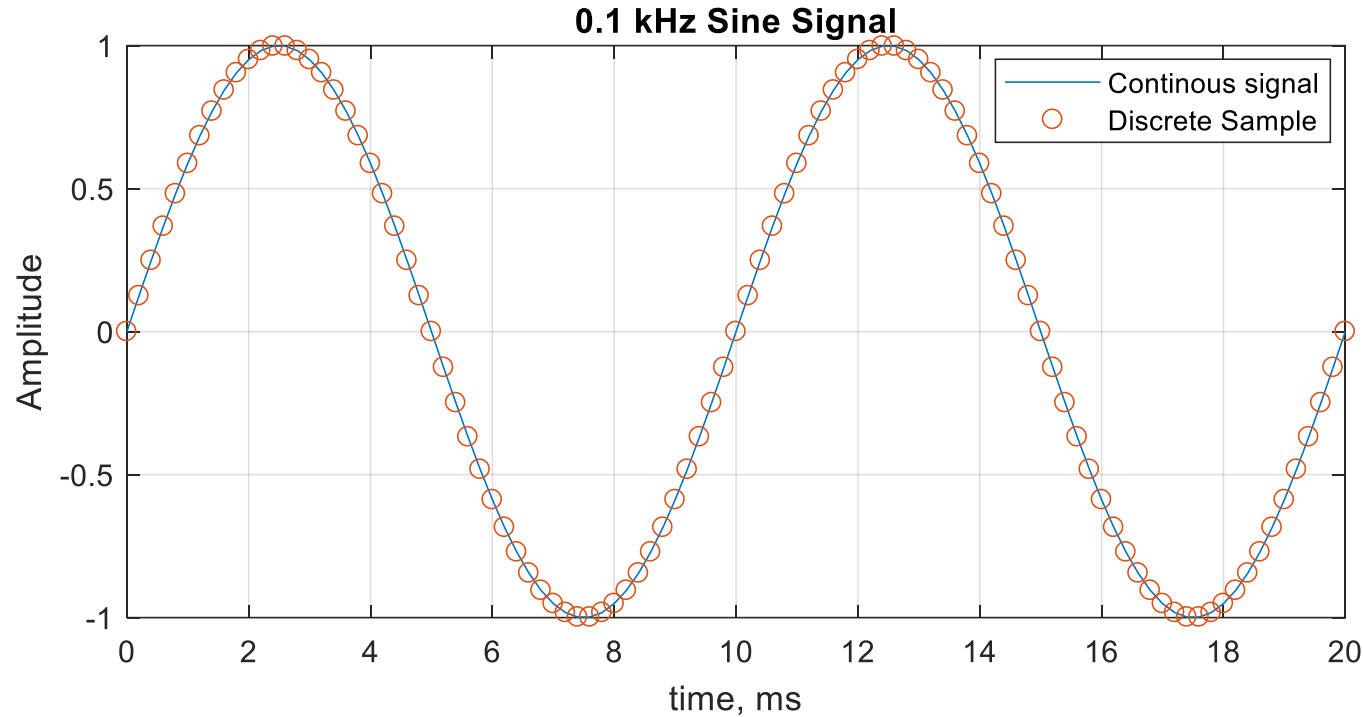
1.3 Generate the Signal

Next is to generate the signal based on the discrete equation of $x[n] = A \sin 2\pi \frac{F}{F_s} n + \phi$. For now, sets the phase $\phi = 0$.

```
x = A*sin(2*pi*(F/Fs)*n);
```

1.4 Plotting the Signal

```
plot(n/Fs*1000,x,n/Fs*1000,x,'o');
legend('Continuous signal','Discrete Sample')
xlabel('time, ms')
ylabel('Amplitude')
titleString = sprintf('%.1f kHz Sine Signal',F/1000);
title(titleString)
grid on
```



From the plot, it can be observed that there are 50 samples at every cycle of the sine signal. Thus, the frequency according to the 50 samples is $\frac{1}{50} = 0.02$. This is also called the frequency ratio f , which is the ratio of $f = \frac{F}{F_s}$. Thus, the discrete signal can now describe as:

$$x[n] = A \sin(2\pi f n + \phi)$$

2.0 Sampling

```

D = 200*1e-3;           % Duration of 200ms

Fs = [100 210 500 2000];
for k = 1:4
    FsSelect = Fs(k);
    N = D*FsSelect;      % Number of samples for duration D.
    n = 0:N;             % Discrete-time (sample).
    x = sin(2*pi*(100/FsSelect)*n);

    subplot(2,2,k), plot(n/FsSelect*1000,x,'-o');
    xlabel('time, ms')
    ylabel('Amplitude')
    titleString = sprintf('Sampled at Fs = %d Hz',FsSelect);
    title(titleString)
    grid on
end
    
```

2.1 Introduction

According to Nyquist theory, a discrete signal will have enough samples when $F_S \geq 2F_N$. Here, F_S is the frequency sampling and F_N is the maximum frequency of the signal.

Now, let's consider a 100Hz sine signal that will be sampled at several sampling frequencies. Note that we set the amplitude as $A = 1$ and duration of the signal at $0 \leq t \leq 200ms$.

2.2 Setting the Time Duration

```
D = 200*1e-3;           % Duration of 200ms
```


2.3 Loop for Four F_s Values

To repeat the process for several F_s values, the four F_s values are set into a single vector. Then for loop is use for the repetition operation.

```
Fs = [100 210 500 2000];  
for k = 1:4
```

2.4 Sampling the Time.

Current F_s value is first assign to variable FsSelect.

```
FsSelect = Fs(k);  
N = D*FsSelect;      % Number of samples for duration D.  
n = 0:N;              % Discrete-time (sample).
```

2.5 Generate the 100Hz Signal

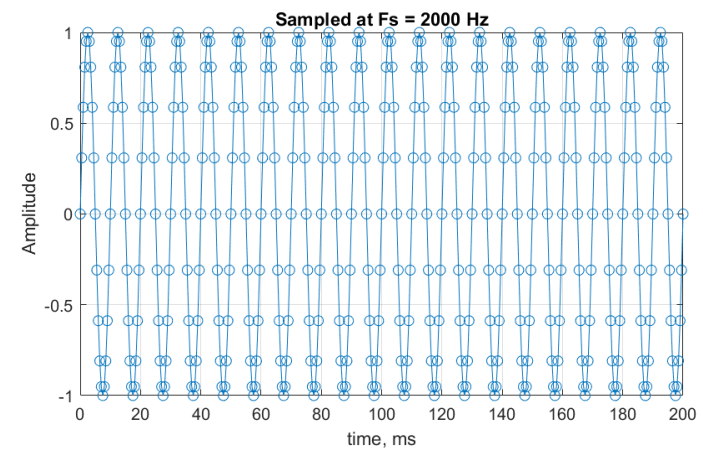
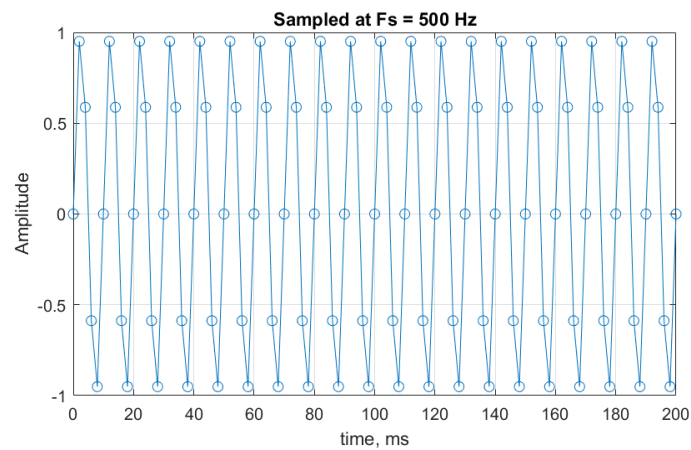
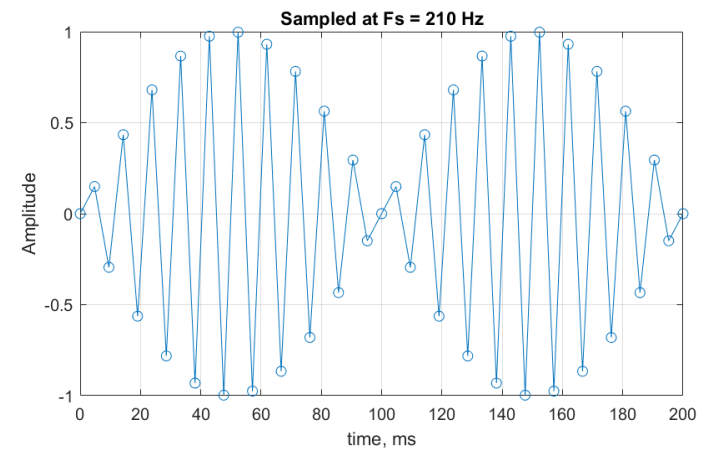
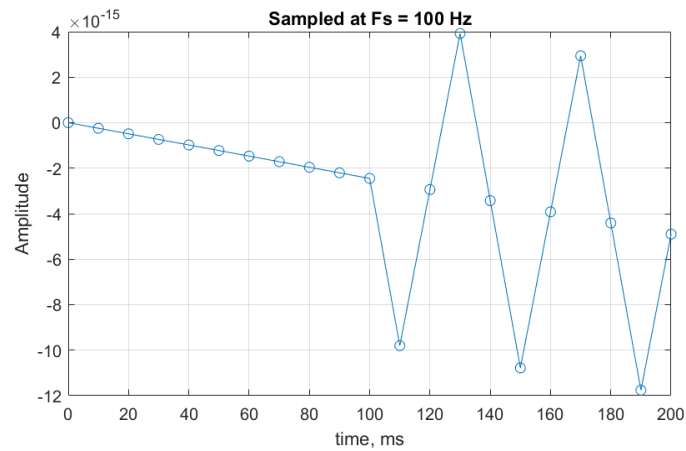
The signal is generated according to $x = A \sin\left(2\pi \frac{F}{F_s} n + \phi\right)$ where $A = 1$, $F = 100\text{Hz}$, $\phi = 0$ and F_s is referring to variable `FsSelect`.

```
x = sin(2*pi*(100/FsSelect)*n);
```

2.6 Plot the Signal

Subplot is use to display all of the four plots on a single window.

```
subplot(2,2,k), plot(n/FsSelect*1000,x, '-o');
xlabel('time, ms')
ylabel('Amplitude')
titleString = sprintf('Sampled at Fs = %d Hz',FsSelect);
title(titleString)
grid on
end
```



2.7 Observation

From the plots, the 100Hz sine signal generated based on F_s below 200Hz (Nyquist frequency) is far from the 100Hz signal should look like. This is where aliasing occurred.

For $F_s = 200\text{Hz}$, the cycle of 100Hz can be seen but with distorted amplitude. Since the frequency is preserved, no aliasing for this signal.

Then, increasing the F_s to 500Hz and 2000Hz increases the detail of the generated signal.

3.0 Discrete-Time Signal Representation

```
n = [0 1 2 3 4 5];
x = sin(0.5*pi*n)
```

```
stem(n,x)
xticks(n)
xlabel('time, n')
ylabel('Amplitude')
```

```
Fs = 2000;           % Frequency sampling in Hz
t  = n/Fs;           % Conversion to t
stem(t,x)
xticks(t)
xlabel('time, t(s)')
ylabel('Amplitude')
```

3.1 Introduction

Since discrete-time signal is constructed based on samples, stem function is used to better represents the discrete-time signal.

Next is how the stem function is written. In this plotting, the x-axis will be the sample number in integer value. Since n is sample, there is no unit for the parameter.

However, plotting the discrete-time signal based on continuous-time t is also possible when the frequency sampling F_s is known.

3.2 Discrete-Time Signal as a Vector

In MATLAB®, discrete signal is represented as a vector. For example, signal $x = \sin(0.5\pi n)$ for $0 \leq n \leq 5$ is having 6 samples (sample no. 0 to sample no. 5). Thus, in MATLAB®, x is a vector with 6 elements. The following code shows that the value for elements 1, 3, and 5 of vector x is 0 while the elements 2, 4 and 6 has the value of 1.

Note that in MATLAB® vector, numbering for element always start with 1. This numbering is called index (position of the element). Note that both number of elements for x-axis vector and y-axis vector must be the same.

```
n = [0 1 2 3 4 5];    % x-axis: Discrete-time
x = sin(0.5*pi*n)      % y-axis: Signal's amplitude
```

$x =$

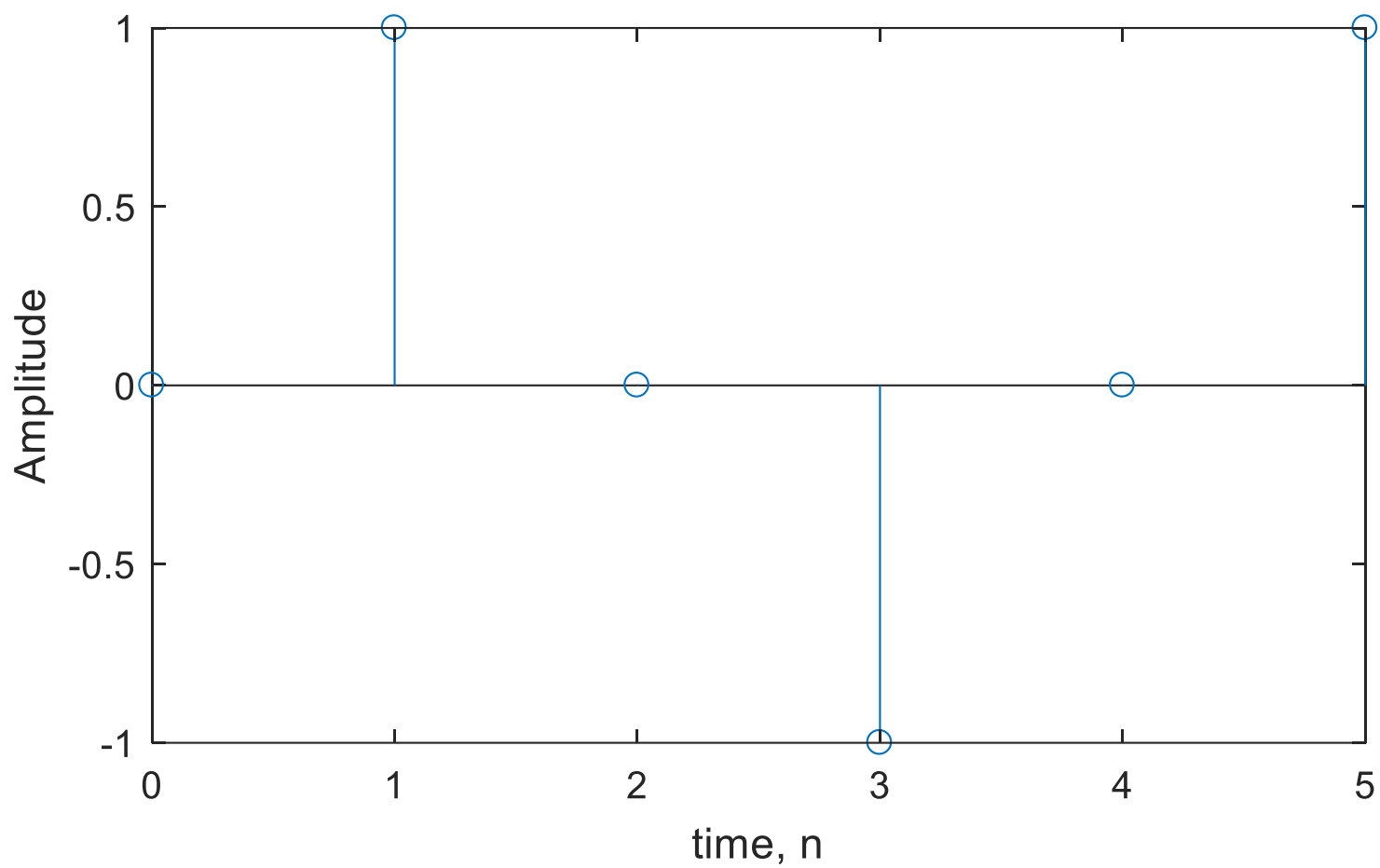
0	1.0000	0.0000	-1.0000	-0.0000	1.0000
---	--------	--------	---------	---------	--------

3.3 Displaying Discrete-Time Signal

Generally, stem function plot discrete sequence data. Since the discrete signal is also a discrete sequence data, then stem function is more appropriate compared to plot function.

Like plot function, the first input to the stem function is the x-axis values, which is the vector n and the second input is the y-axis value, the vector x .

```
stem(n,x)
xticks(n)
xlabel('time, n')
ylabel('Amplitude')
```

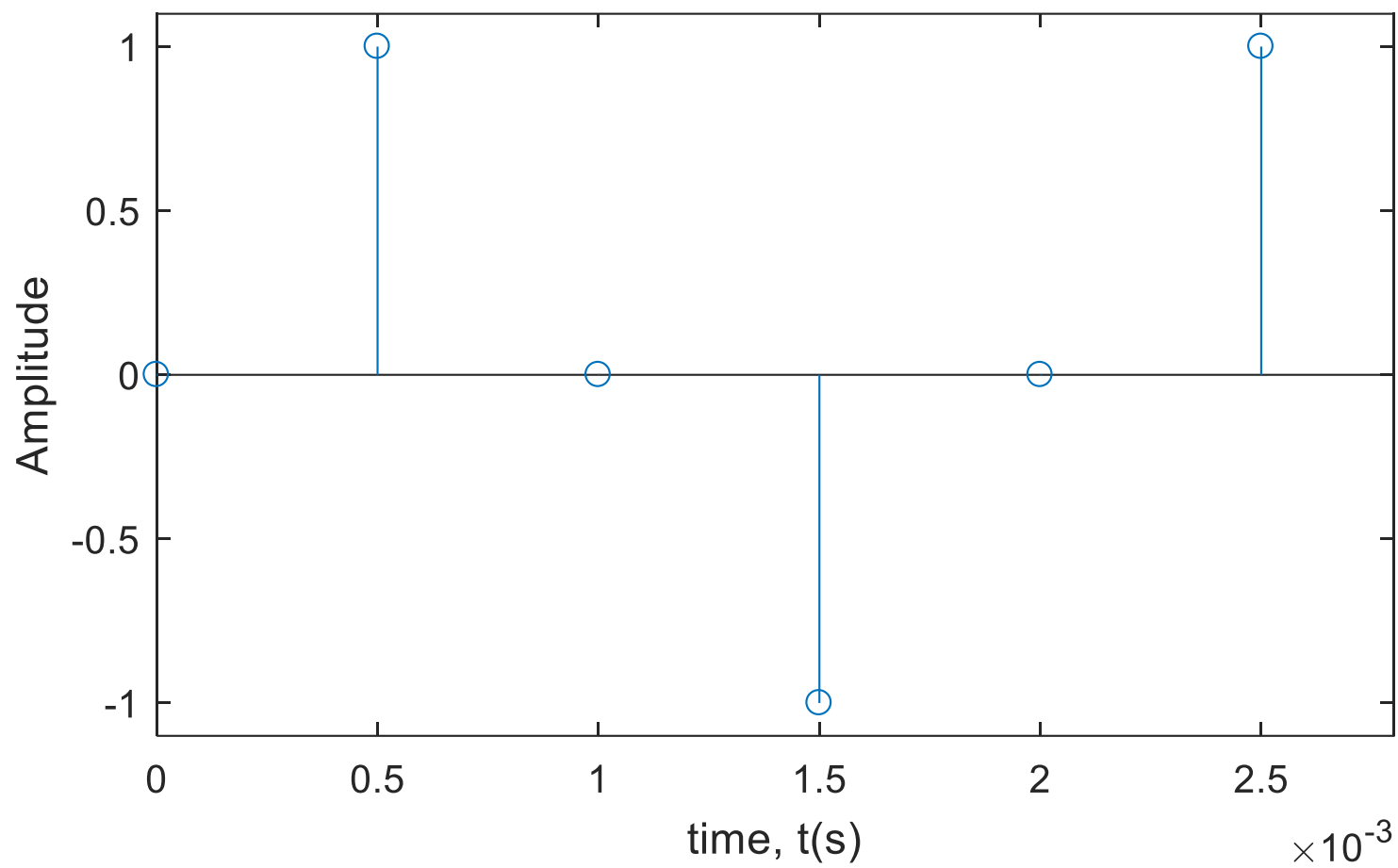
3.4 Displaying Discrete-Time Signal with Continuous-Time t

In signal processing, prior to the discrete-time signal, originally the signal is in continuous-time form. The process of converting from the continuous-time to the discrete-time signal is called sampling. If the sampling frequency F_s is provided, the discrete-time signal can be displayed with the continuous-time t instead of the discrete-time n .

Now let's assume $F_s = 2\text{kHz}$. Then, the discrete-time n can be converted into continuous-time t according to $t = \frac{n}{F_s}$.

```

Fs = 2000;           % Frequency sampling in Hz
t  = n/Fs;           % Conversion from n to t
stem(t,x)
xticks(t)
xlabel('time, t(s)')
ylabel('Amplitude')
    
```



4.0 Specifying Time (n) Vector

Every 1-dimesional signal comes with two quantities, the amplitude and time. When plotting, amplitude will be on the y-axis and time on the x-axis. As previously shown, discrete-time signal represents these two quantities as vectors.

The following sub-sections demonstrate how to generate and plot a discrete-time signal shown below with different time vector. Try to figure out how many samples does the signal have.

$$x = [2, 1, 0, -2, -1, 0, 2, 1]$$

4.1 Signal with Time Start at $n \neq 1$

For example, signal $x = [2, 1, 0, -2, -1, 0, 2, 1]$ is known to have the time start at $n = 0$. Thus, the time duration for this signal is $0 \leq n \leq 7$.

On another example, signal $x = [2, 1, \underline{0}, -2, -1, 0, 2, 1]$ starts at $n = -2$ and ends at $n = 5$. Thus, the time duration for signal x is $-2 \leq n \leq 5$.

Since the discrete-time n is a sequence of integer value by increment of 1, colon (:) operator is a way to simplify the code.

```
x = [2, 1, 0, -2, -1, 0, 2, 1]
n1 = 0:7
n2 = -2:5
```

```
n1 =
    0     1     2     3     4     5     6     7

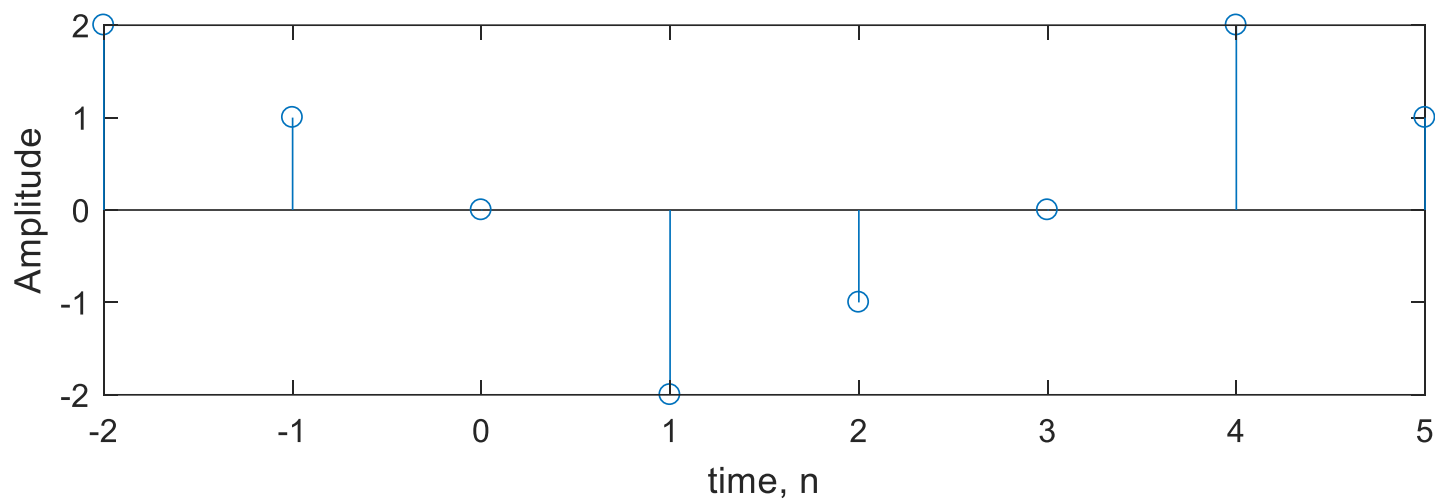
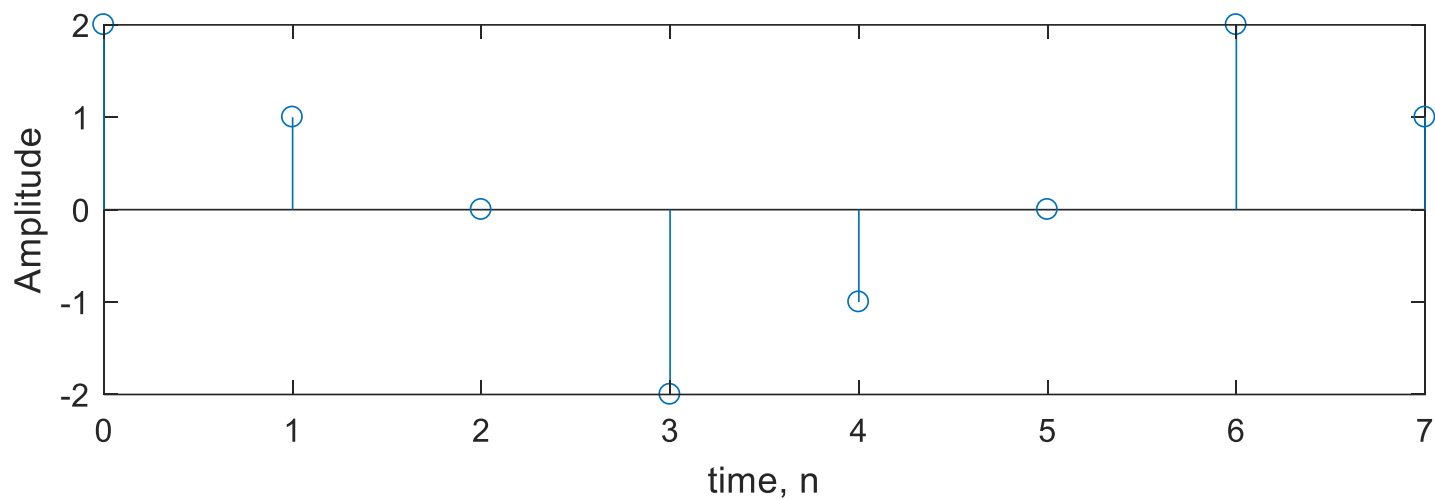
n2 =
   -2    -1     0     1     2     3     4     5
```

4.2 Plotting the Signal

Then, generating and plotting the signal are similar to the previous examples

```
subplot(2,1,1), stem(n1,x);  
xticks(n1)  
xlabel('time, n')  
ylabel('Amplitude')
```

```
subplot(2,1,2), stem(n2,x);  
xticks(n2)  
xlabel('time, n')  
ylabel('Amplitude')
```



4.3 Signal with Time Start at $n = 1$

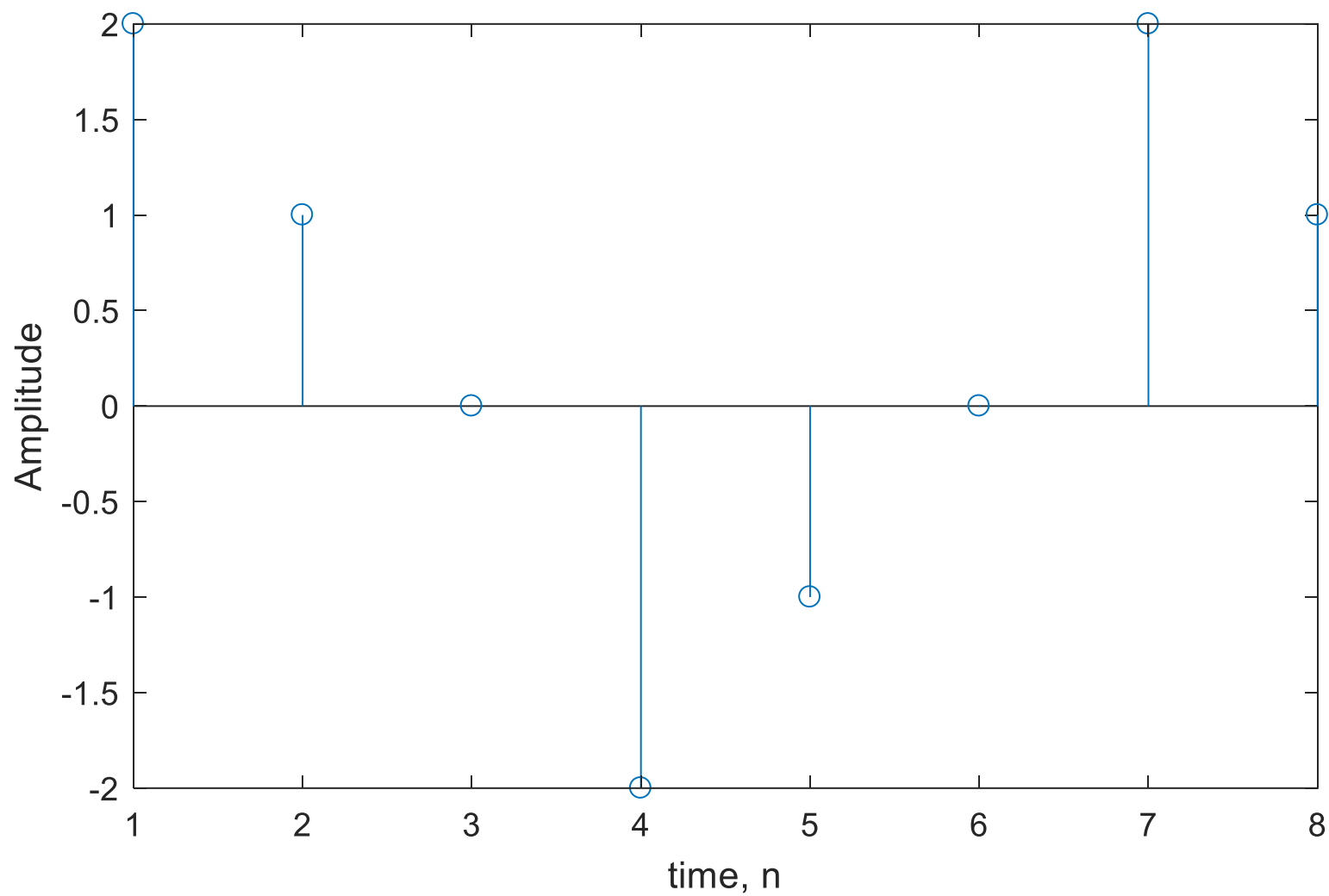
When a discrete-time signal start at $n = 1$, there is an option to simplify the code. This is because vector indexing in MATLAB® also starts at 1. Thus, we can skip specifying the time vector since MATLAB® will use the vector indices as the time vector.

Now let's repeat the previous example but with time start at $n = 1$. In this case, we only need to specify the vector x and skip specifying the time vector.

When using the stem function, we can also skip the time input.

```
x = [2,1,0,-2,-1,0,2,1];
```

```
figure, stem(x);  
xticks(1:8)  
xlabel('time, n')  
ylabel('Amplitude')
```

5.0 Arithmetic Operation

This example will demonstrate performing four arithmetic operations, the add, minus, multiply and divide operations. For simplicity, the operations are performed between two signals. However, it will also work for more than two signals.

Let's assume we have two discrete-time signals as the following, performing the arithmetic operation between the two signals is discussed on the following sub-topics.

$$x_1 = [1, 1, 2, 2] \text{ for } 0 \leq n \leq 3$$

$$x_2 = [1, 2, 1] \text{ for } -1 \leq n \leq 1$$

5.1 Signal Length

Arithmetic operation on two discrete-time signals should follow below rules:

- 1) Both signals are at the same length.
- 2) The operation is done on the same sample no.

Since x_1 and x_2 are not at the same length, we must first set the signal vectors to be the same length by padding with zero. Padding is done by first setting n starting from the lower n and end with the higher n of the two signals. Then pad the zeros to the empty elements.

```
n = [-1 0 1 2 3];
x1 = [ 0 1 1 2 2];    % Pad zero at n=-1
x2 = [ 1 2 1 0 0];    % Pad zero at n=2&3
```

5.2 Add and Minus Operation

With both signals are now having the same length, performing the arithmetic operation is as the following where operators +, and - are used to perform plus and minus operation respectively.

```
xplus      = x1+x2
xminus     = x1-x2
```

xplus =

1 3 2 2 2

xminus =

-1 -1 0 2 2

5.3 Multiply and Divide Operation

Multiply and divide between two signals use `.*` and `./` operators respectively. Note that the operators are preceded with period (.) operator. These are called array operator. Without the period operator, the operation will perform matrix operation and output a different value. For add and minus operation, the period operator is not compulsory because both array and matrix operation for add and minus result similar values.

```
xmultiply = x1.*x2
xdivide   = x1./x2
```

```
xmultiply =
```

```
      0      2      1      0      0
```

```
xdivide =
```

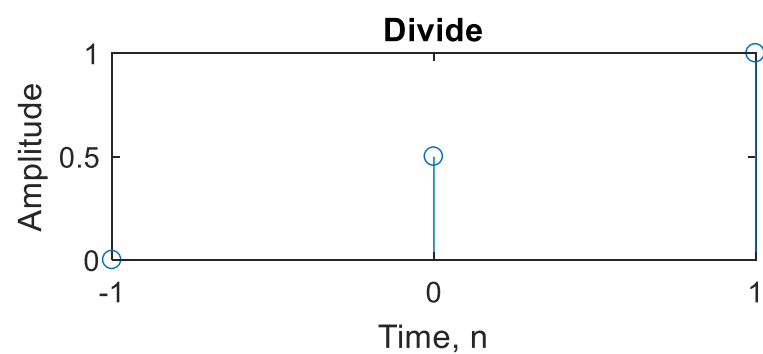
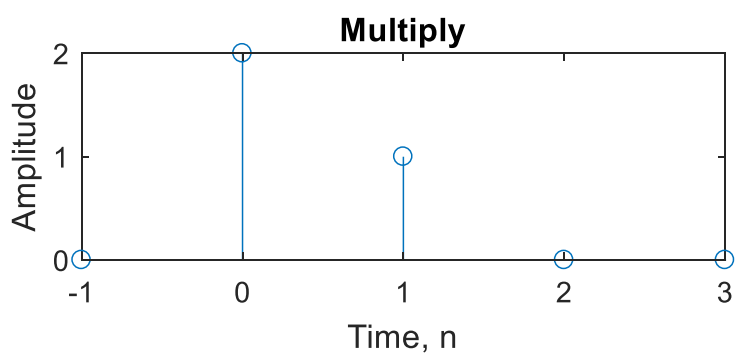
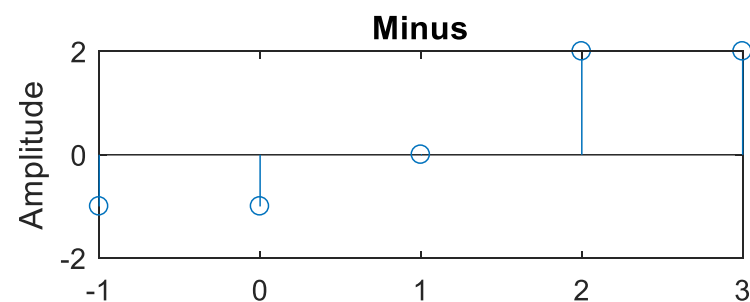
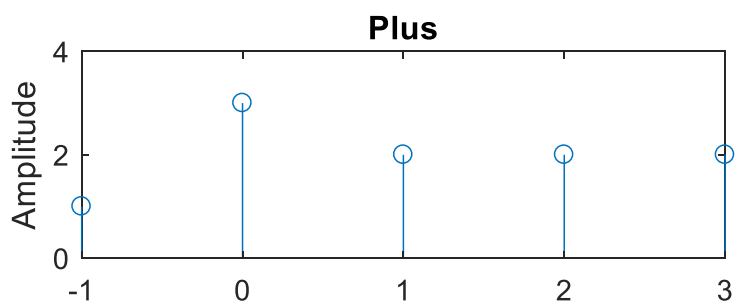
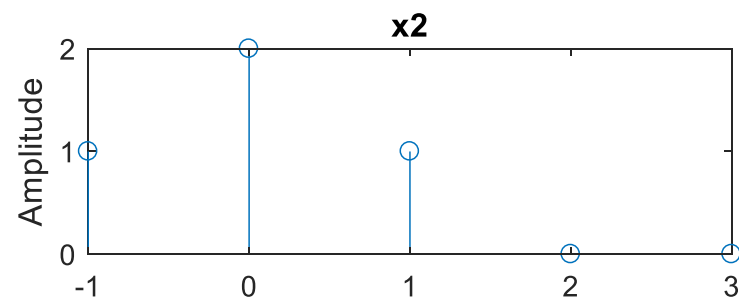
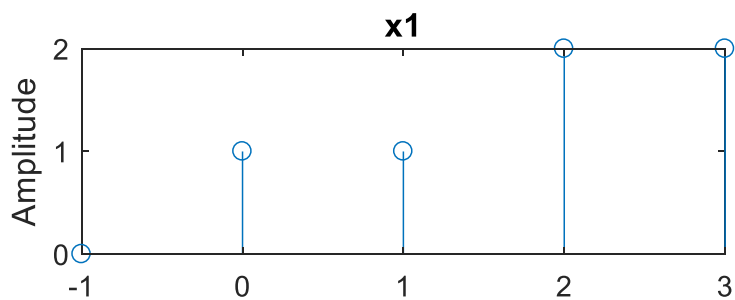
```
      0    0.5000    1.0000      Inf      Inf
```

5.4 Displaying the Results

In this example, all signals are combined into a single array called 'data'. By having the array, plotting the six signals is simplified by using a for loop. For the same reason, the titles of the six signals are combined into a single cell array called 'str'. Refer to MATLAB® documentation on the difference between array and cell array.

```
data = [x1;x2;xp1us;xminus;xmultiply;xdivide];
str = {'x1','x2','Plus','Minus','Multiply','Divide'};

for k = 1:6
    subplot(3,2,k), stem(n,data(k,:))
    xticks(n)
    xlabel('Time, n')
    ylabel('Amplitude')
    title(str{k})
end
```



6.0 Folding Operation

```

n = -1:3                                % Original time vector
x = [3 5 1 -1 2]                        % Original amplitude vector

nfold = sort(-n)                        % Folded time vector
idxfold = length(x):-1:1                % Reversing index
xfold = x(idxfold)                     % Folded amplitude vector

subplot(2,1,1), stem(n,x)
title('x[n]')
xticks(n)
xlabel('Time, n'), ylabel('Amplitude')

subplot(2,1,2), stem(nfold,xfold)
title('x[-n]')
xticks(nfold)
xlabel('Time, n'), ylabel('Amplitude')
    
```


6.1 Introduction

Folding operation is define as $x'[n] = x[-n]$. Since there are two vectors for any given signal (time vector and the amplitude vector), we need to fold both vectors when performing the folding operation.

In this example, let's define a discrete-time signal as vector $x = [3, \underline{5}, 1, -1, 2]$ where $n = 0$ is position at element no. 2 (index 2). Thus, the time and amplitude vectors of the signal can be written as below:

```
n = -1:3                                % Original time vector
x = [3 5 1 -1 2]                        % Original amplitude vector
```

```
n =
    -1     0     1     2     3
```

```
x =
     3     5     1    -1     2
```

6.2 Fold the Time Vector n

Folding operation is where $y[n]=x[-n]$. Thus, folding the time vector is simply assigning negative value to the n.

Then, since time vector is a sequence of an increasing integer number, sort is applied.

```
nfold = sort(-n)           % Folded time vector
```

```
nfold =
```

```
    -3    -2    -1     0     1
```

6.3 Folding the Amplitude Vector

Folding the amplitude vector is simply to reverse the indexing of the vector and rearrange the vector based on the reverse indexing.

For example, reverse indexing of $1, 2, 3, \dots, N$ is $N, \dots, 3, 2, 1$. Note that indexing of vector in MATLAB® always start at 1 and end at the length N of the vector. Thus, reversing the indices can be done using 2 colon operator ($:$) with increment of -1 such that $N:-1:1$.

```
idxfold = length(x):-1:1    % Reversing index
xfold = x(idxfold)          % Folded amplitude vector
```

idxfold =

5 4 3 2 1

xfold =

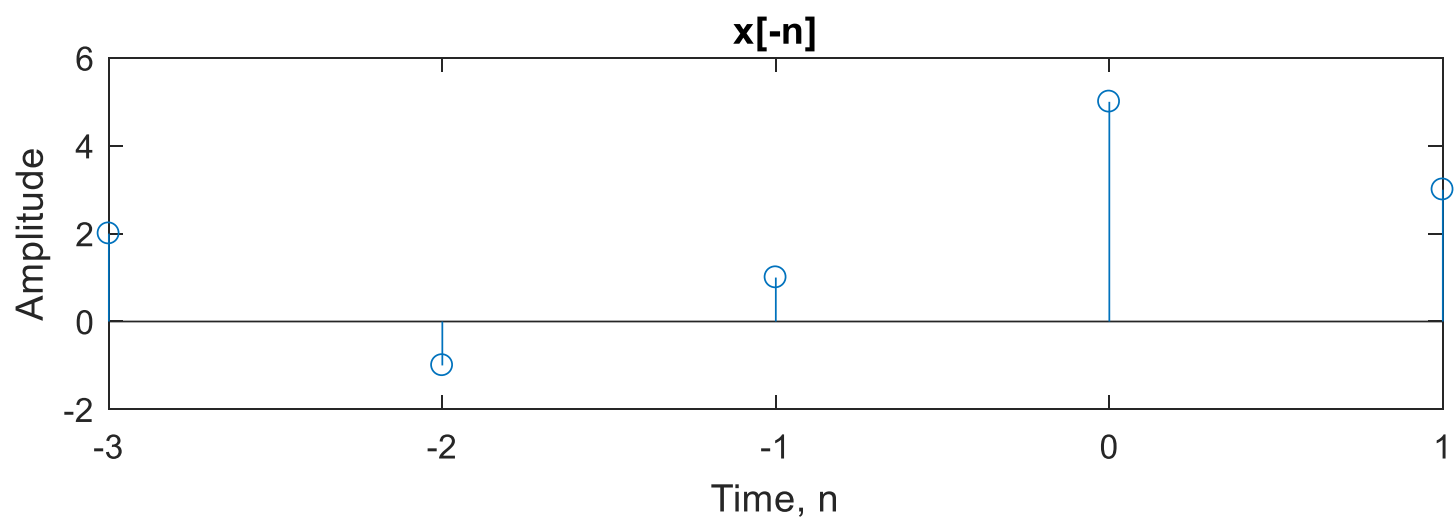
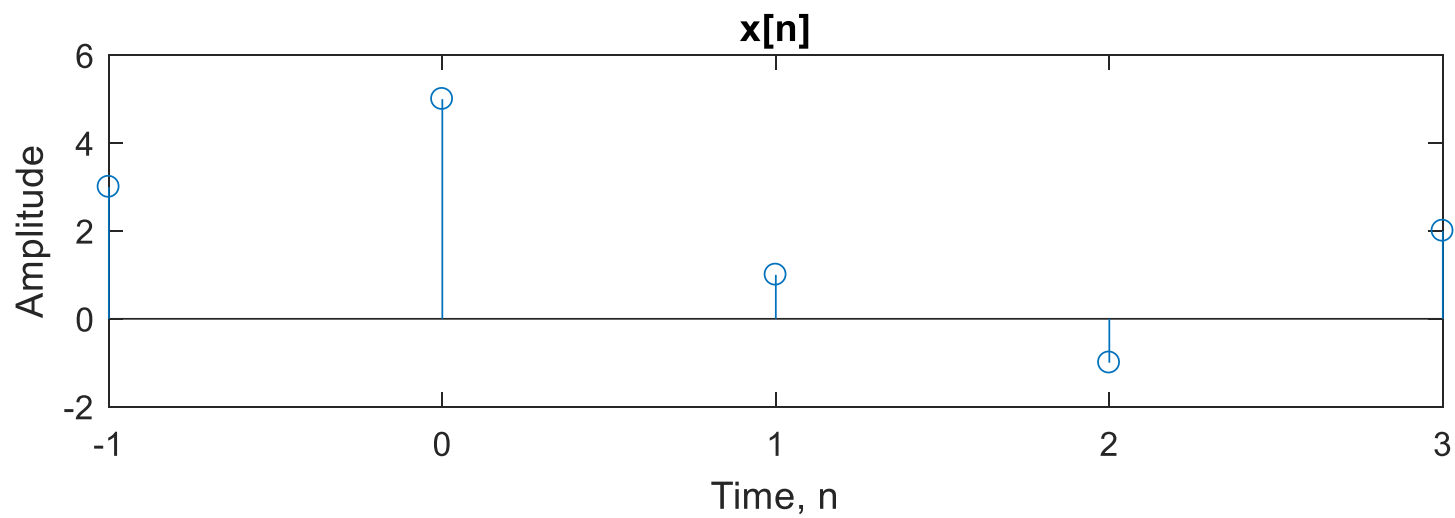
2 -1 1 5 3

6.4 Plotting the signal

Below is the resulting folded signal when plotted.

```
subplot(2,1,1), stem(n,x)
title('x[n]')
xticks(n)
xlabel('Time, n')
ylabel('Amplitude')

subplot(2,1,2), stem(nfold,xfold)
title('x[-n]')
xticks(nfold)
xlabel('Time, n')
ylabel('Amplitude')
```



7.0 Delay Operation

```

n = 2:7
x = sin(0.2*pi*n)
d1 = 2;      nd1 = n+d1      % Shift to the right
d2 = -3;     nd2 = n+d2     % Shift to the left
xd1 = x
xd2 = x
na11 = [n;nd1;nd2];
str = {'x[n]', 'x[n-2]', 'x[n+3]'};
for k = 1:3
    subplot(3,1,k), stem(na11(k,:),x)
    xlim([-3 10])
    xticks(-3:10)
    xlabel('Time, n')
    ylim([-1.5 1.5])
    ylabel('Amplitude')
    title(str{k})
end
    
```

7.1 Introduction

Delay operation is defined as $x'[n]=x[n-d]$. Delay will shift the signal either to the right or left by d samples.

In this example, we will use the following signal to demonstrate the delay operation.

```
n = 2:7
x = sin(0.2*pi*n)
```

n =						
	2	3	4	5	6	7
x =						
	0.9511	0.9511	0.5878	0.0000	-0.5878	-0.9511

7.2 Delaying the Time Vector

We can rewrite the equation as $x'[n]=x[M-d]$ where input signal at time $M-d$ are mapped to output signal at time n . Delaying the time vector means finding the output signal time range. Time range start from the first nonzero sample and end at the last nonzero sample. This can be simply found by mapping the input signal time range M to output signal time range, which lead us to $x'[n+d]=x[M]$. Thus, $M=n+d$. In below code, M is assign as $nd1$ and $nd2$.

```
d1 = 2;      nd1 = n+d1      % shift to the right
d2 = -3;     nd2 = n+d2     % shift to the left
```

```
nd1 =
     4     5     6     7     8     9
```

```
nd2 =
    -1     0     1     2     3     4
```


7.3 Delaying the Amplitude Vector

For delay operation, the samples sequence of the amplitude vector does not change. Thus, the input and output of the delay operation are exactly the same for the amplitude vector.

$$xd1 = x$$

$$xd2 = x$$

$$xd1 =$$

0.9511 0.9511 0.5878 0.0000 -0.5878 -0.9511

$$xd2 =$$

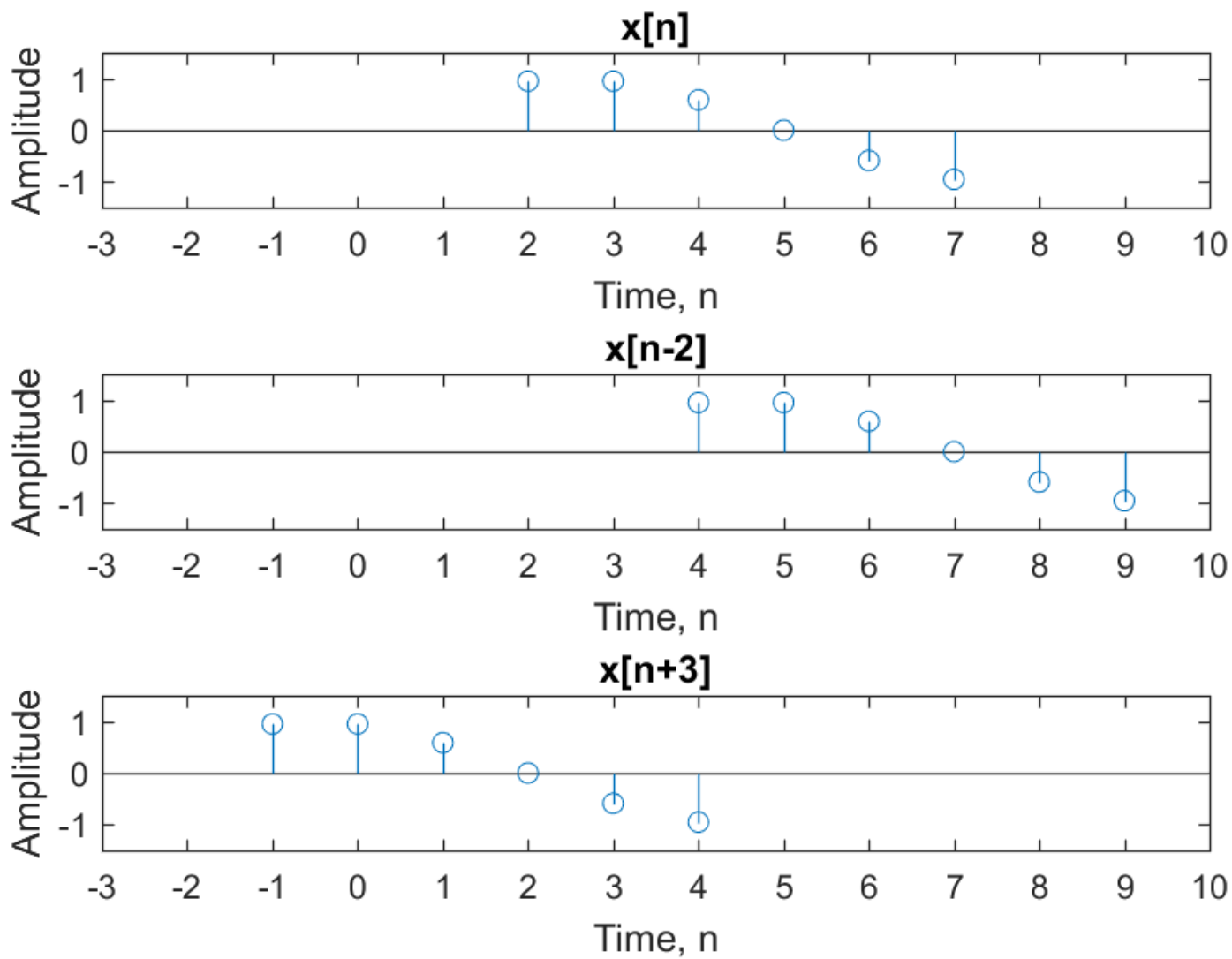
0.9511 0.9511 0.5878 0.0000 -0.5878 -0.9511

7.4 Plotting the Signal

In the plotting, `xlim()` is use to control the x-axis range, which give us better view on the delay operation.

```

na11 = [n;nd1;nd2];
str = {'x[n]', 'x[n-2]', 'x[n+3]'};
for k = 1:3
    subplot(3,1,k), stem(na11(k,:),x)
    xlim([-3 10])
    xticks(-3:10)
    xlabel('Time, n')
    ylim([-1.5 1.5])
    ylabel('Amplitude')
    title(str{k})
end
    
```



8.0 Generalizing Discrete-Time Operation

```

n = 2:6
x = 2*sin(0.3*pi*n)

[nout,y1] = foldelay(n,x,-2,-1)    % y1[n]=x[2-n]
plotsignal([n;nout],[x;y1])

[nout,y2] = foldelay(n,x,-2,-1)    % y2[n]=x[-n-2]
plotsignal([n;nout],[x;y2])

% Function to fold and delay a signal
function [M,xout]=foldelay(n,x,d,f)
    M = sort((n+d)*f)
    s = n(1)-1;
    xout = x(f*M-d-s)
end
    
```

```

% Function to display the result.
function plotsignal(n,x)
    str = {'x[n]','y[n]'};
    for k = 1:2
        subplot(3,1,k), stem(n(k,:),x(k,:))
        xlim([-7 7])
        xticks(-7:7)
        xlabel('Time, n')
        ylabel('Amplitude')
        title(str{k})
    end
end
    
```

8.1 Introduction

Here, we will discuss examples of when folding and delay operation occurs in single operation. Let's consider the following operation on signal $x[n]=2\sin(0.3\pi n)$ for $1 < n < 7$ for the discussion:

$$y_1[n] = x[2 - n]$$

$$y_2[n] = x[-n - 2]$$

```
n = 2:6
x = 2*sin(0.3*pi*n)
```

n =

2 3 4 5 6

x =

1.9021 0.6180 -1.1756 -2.0000 -1.1756

8.2 Example 1: $y[n] = x[2 - n]$

To do the operation, this example called a generalized function by specifying delay amount d and folding status f (fold: $f=-1$, no fold: $f=1$). In this example, $d=-2$ and $f=-1$.

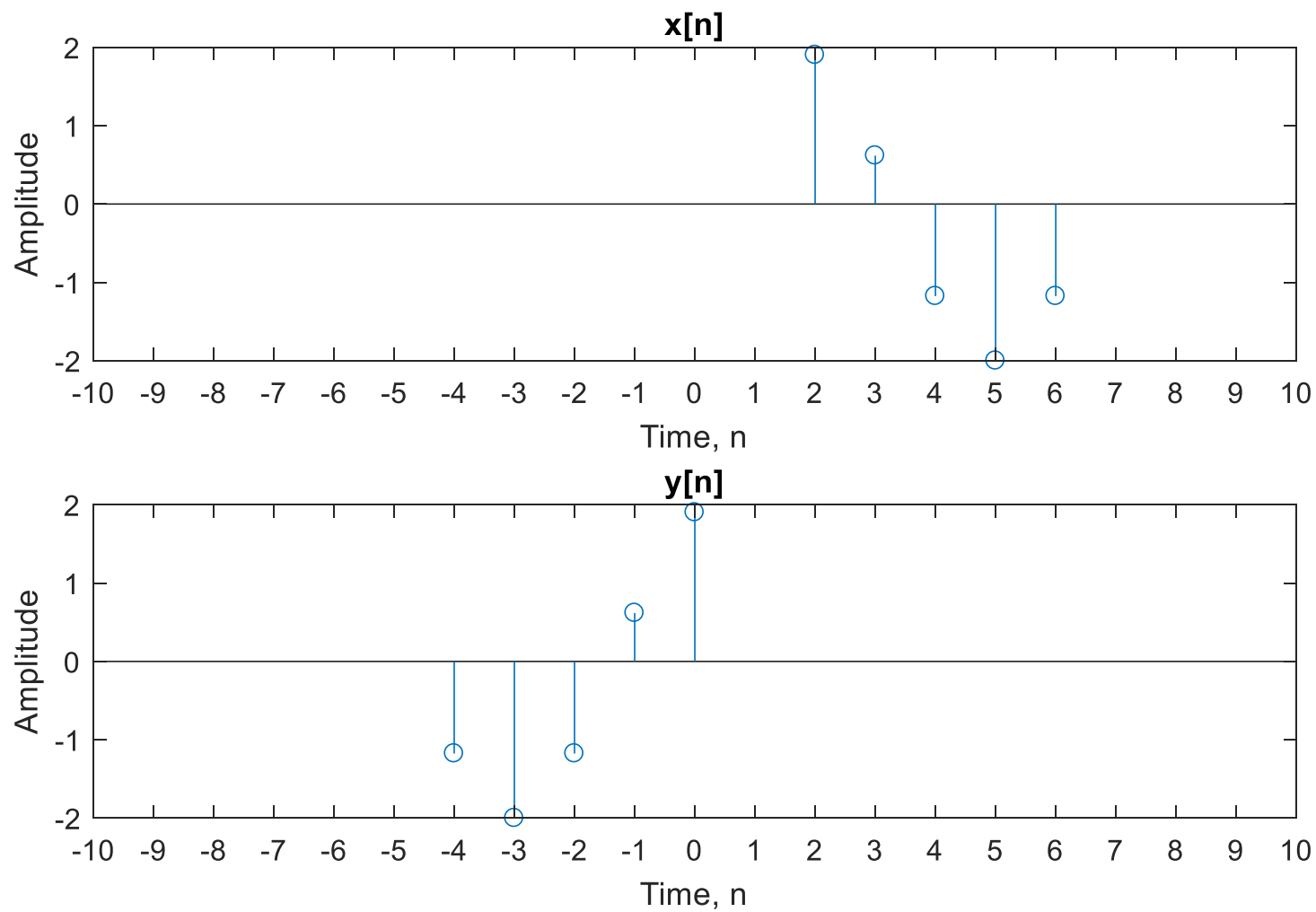
```
[nout,y1] = foldelay(n,x,-2,-1)
plotsignal([n;nout],[x;y1])
```

nout =

-4 -3 -2 -1 0

y1 =

-1.1756 -2.0000 -1.1756 0.6180 1.9021



8.3 Example 2: $y_2[n] = x[-n - 2]$

Similar to Example 1, function `foldelay` is called by specifying delay amount `d` and folding status `f` (fold: `f=-1`, no fold: `f=1`). In this example, `d=2` and `f=-1`.

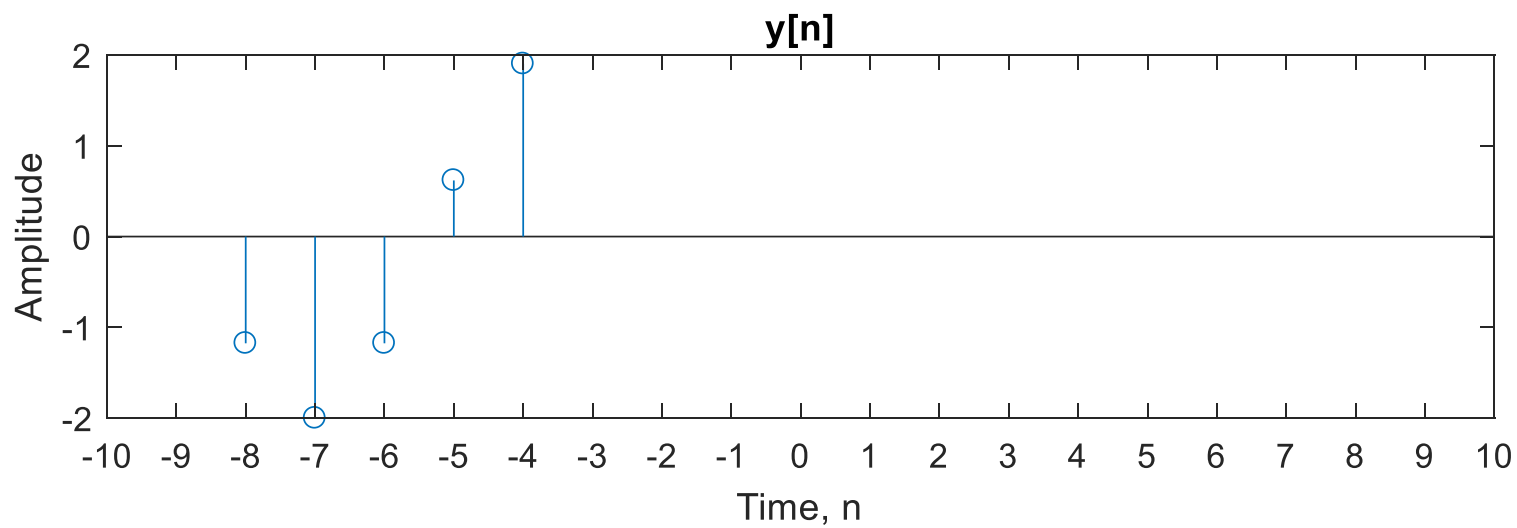
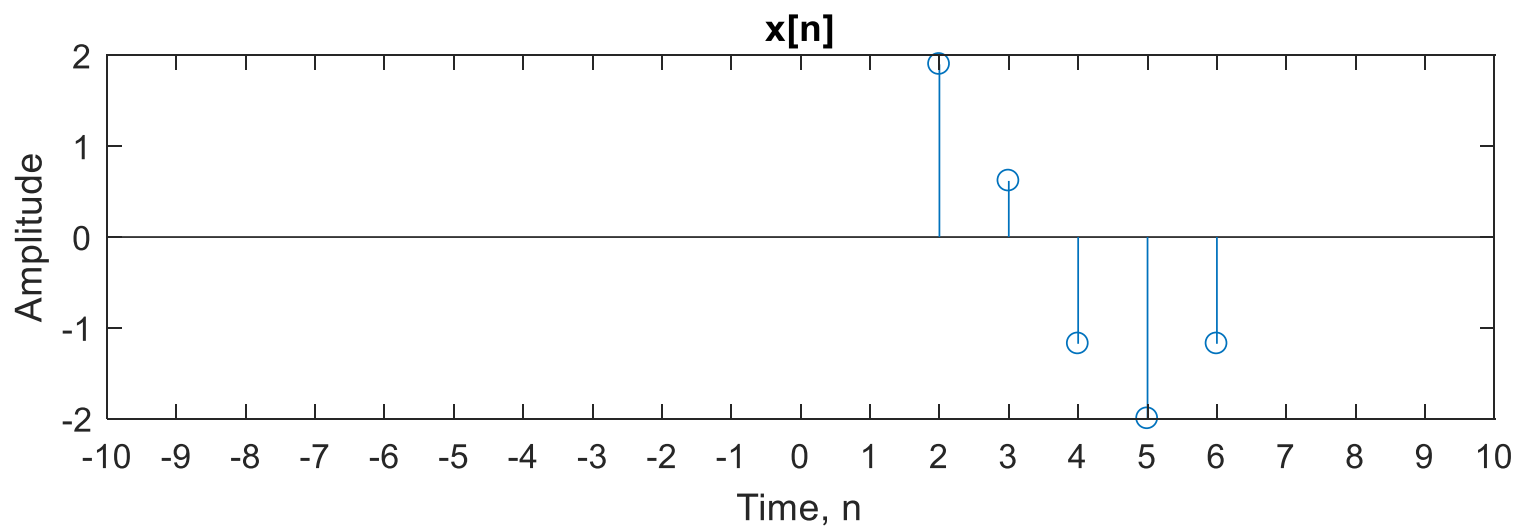
```
[nout,y2] = foldelay(n,x,2,-1)
plotsignal([n;nout],[x;y2])
```

`nout` =

-8 -7 -6 -5 -4

`y2` =

-1.1756 -2.0000 -1.1756 0.6180 1.9021



8.4 Generalizing Folding and Delay Operation

In this example, a function is created to perform any given folding and delay operation to a signal. The function name is `foldelay` and it takes four inputs; n (signal time vector), x (signal amplitude vector), d (delay amount) and f (folding status). Then, it return two outputs; M (output time vector) and x_{out} (output amplitude vector).

```
function [M,xout]=foldelay(n,x,d,f)
```

8.5 Operation on Time Vector

Folding and delay operation can be generalized as $y[n] = x[fM - d]$. $f=-1$ is set when folding occurs and $f=1$ is for no folding. Then, d is the delay amount. Thus, the output time vector is $M = (n - d) \times f$. Please recall to Section 6 on the usage of sort function.

```
M = sort((n+d)*f)
```

8.6 Operation on Amplitude Vector

For amplitude vector, we need to formulate the right indexing to access the input vector. In this case, accessing elements in vector x .

If equation $y[n] = x[fM - d]$ is directly implemented, there is possibility the index $fM - d$ will becomes ≤ 0 . This will cause an error in MATLAB[®] since the lowest indexing value allowed is 1. Thus, we need to shift the indexing to a start value of 1.

Shifting the indexing to a start value of 1 can be done by first finding the distance between the lowest input time in n towards value 1. Then, shift the distance amount s in the operation such that $y[n] = x[fM - d - s]$.

```
s = n(1)-1;
xout = x(f*M-d-s)
```

```
end
```

8.7 Plotting the Signal

Below is the function called from Example 1 and Example 2 to plot the signals of before and after the operation. Note that this function does not have an output argument since none is required.

```
function plotsignal(n,x)
    str = {'x[n]','y[n]'};
    for k = 1:2
        subplot(3,1,k), stem(n(k,:),x(k,:))
        xlim([-7 7])
        xticks(-7:7)
        xlabel('Time, n')
        ylabel('Amplitude')
        title(str{k})
    end
end
```